

Webhook 보안 검증 _verifySignature 구현

HMAC-SHA256 · timingSafeEqual · rawBody Buffer

강의 25분 + 실습 40분

위협 모델 4가지

_verifySignature 완전 해부

실습 4시나리오

이 세션이 답하는 질문

THREE KEY QUESTIONS — SECURITY DECISIONS EXPLAINED

Q1. 서명 검증을 왜 `===`로 비교하면 안 되는가?

일반 비교는 첫 번째 다른 바이트에서 즉시 종료 → 응답 시간으로 올바른 서명을 한 자리씩 추측 가능 (Timing Attack)

→ `timingSafeEqual` : 길이가 같으면 항상 동일 시간에 비교

Q2. `rawBody`를 `JSON.parse` → `JSON.stringify` 후 서명 계산하면 안 되는가?

키 순서 변경 · 공백 제거로 한 바이트라도 달라지면 HMAC 결과가 완전히 달라짐

→ 반드시 원본 `Buffer(rawBody)` 그대로 계산

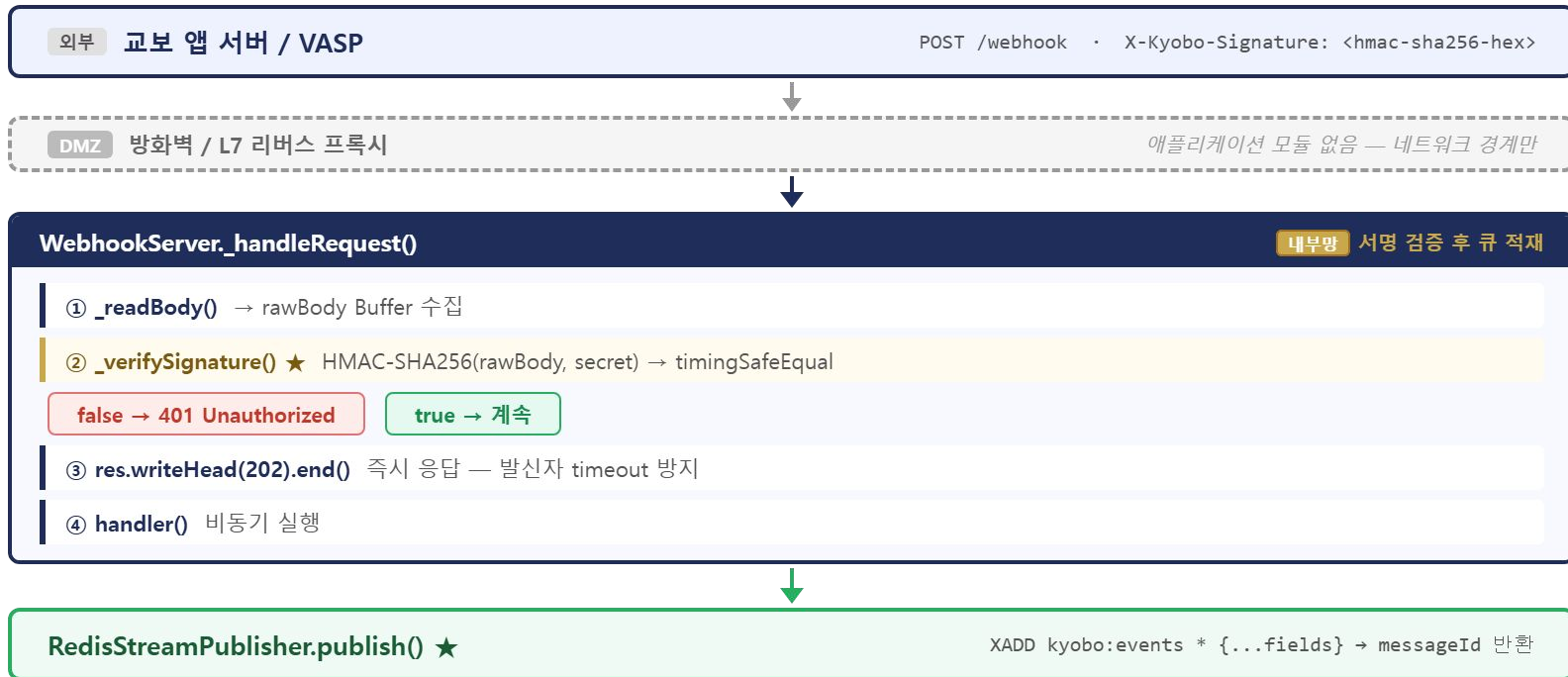
Q3. `timingSafeEqual`이 throw하는 경우는?

두 Buffer의 길이가 다를 때 `TypeError: Input buffers must have the same byte length` 발생

→ `if (sigBuf.length !== expBuf.length) return false;` 로 사전 차단

전체 흐름에서의 위치

내부망 수신 경계 — SIGNATURE VERIFIED BEFORE QUEUE ENQUEUE



Webhook 인증 문제의 본질

WHY CRYPTOGRAPHIC SIGNATURE IS THE ONLY TRUST BASIS

S5에서 WebhookServer가 202를 반환하고 Queue에 적재하는 구조를 배웠다. 그런데 한 가지 질문이 남는다.

교보 앱 서버만 이 Webhook을 호출해야 한다.

그런데 공격자가 같은 엔드포인트로 HTTP POST를 보내면?

→ 서버는 합법적인 요청인지, 공격인지 어떻게 구분하는가?

✗ 불충분한 방어 수단

- HTTP 엔드포인트는 공개되어 있다
- IP 필터링은 VPN·프록시로 우회 가능
- API Key만으로는 payload 변조 탐지 불가
- TLS만으로는 발신자 신원 보장 불가

✓ 유일한 신뢰 기반

- 암호학적 서명 (HMAC)
- secret을 모르면 올바른 서명 계산 불가
- body가 한 바이트라도 바뀌면 서명 불일치
- 발신자 인증 + 본문 무결성 동시 보장

방화벽이 완벽해도 HMAC이 필요한가?

IP-LEVEL TRUST vs REQUEST-LEVEL TRUST — DIFFERENT QUESTIONS

방화벽이 정상 동작한다고 가정 — VASP IP 대역만 내부망 진입 허용. 그래도 HMAC이 필요한 이유는?

방화벽이 답하는 질문

- "이 IP 대역에서 온 요청인가?"
- VASP 허용 IP → 통과
- VASP IP 대역 내 모든 서버를 신뢰
- 페이로드 내용 · 발신 시스템은 확인 안 함

HMAC이 답하는 질문

- "이 secret을 가진 시스템이 서명한 요청인가?"
- VASP 공식 서버만 secret 보유
- 같은 IP 대역이라도 secret 없으면 차단
- 페이로드 1바이트 변조도 감지

방화벽이 버티고 있어도 통과하는 케이스

VASP 내부망의 침해된 다른 서버가 위조 요청 전송

같은 IP → 통과

✓ secret 없음 → 401

전송 중 페이로드 변조 (내부 MITM)

내용 무관 → 통과

✓ 서명 불일치 → 401

방화벽 오설정 · 제로데이 발생

차단 실패

✓ 서명 없음 → 401

결론 — 방화벽과 HMAC은 독립적인 레이어. 방화벽은 IP 대역 단위로 신뢰하고, HMAC은 요청 단위로 신뢰한다. 방화벽이 완벽해도 HMAC 없이는 같은 IP 대역 내 위조 요청을 막을 수 없다.

보안 위협 모델 4가지

THREAT MODEL — WHAT `_verifySignature` DEFENDS AGAINST

위협 1: 위조 요청 (Forgery)

공격자가 임의 payload로 POST 전송

HMAC 없으면 서버가 처리 → 가짜 NFT 발행

→ secret 없으면 올바른 서명 계산 불가 → 401 ☒ (이 세션 구현)

위협 2: 재전송 공격 (Replay Attack)

합법 요청을 캡처해서 나중에 재전송

서명은 맞지만 동일 requestId 재처리 위험

→ timestamp ±5분 체크 + requestId 중복 차단 (S9)

위협 3: 타이밍 공격 (Timing Attack)

응답 시간으로 서명을 한 바이트씩 추측

일반 === 비교는 첫 불일치에서 즉시 종료

→ timingSafeEqual로 항상 동일 시간 비교 ☒

위협 4: 본문 변조 (Tampering)

중간자가 HTTP body를 바꿔서 전달

서명은 원본 body로 계산됨

→ body 변조 시 서명 불일치 → 401 ☒

이 세션 범위: `_verifySignature`가 위협 1, 3, 4를 막는다 · 위협 2는 S9 IdempotencyGuard에서 상세 처리

Webhook 인증 방식 전체 비교

HMAC IS THE INDUSTRY STANDARD — BUT NOT THE ONLY OPTION

방식	특성	Webhook 적합성	사용 예시
단순 해시 (SHA256)	키 없음 → 누구나 계산 가능	❌ 위조 방지 불가	무결성 확인 전용
HMAC-SHA256	대칭키(공유 secret), 빠름, 구현 단순	✅ 업계 표준	GitHub, Stripe, Slack
RSA / ECDSA	비대칭키, 공개키로 누구나 검증 가능	△ 가능, 오버킬	블록체인 TX, EIP-712
IP 화이트리스트	발신 IP 대역 제한, 단독으로 우회 가능	△ 보완 수단	구형 VASP, 내부망
API Key (Bearer)	헤더에 토큰, 단순하지만 노출 위험	△ 단순 인증	내부 시스템 간
mTLS	클라이언트 인증서 상호 검증, 고신뢰	✅ 금융권 고신뢰	은행 간 전문, HSM 구간
VASP 자체 서명	VASP별 독자 방식, 계약 시 스펙 확정	계약 확인 필요	월렛원, Circle 등

왜 HMAC이 기준? GitHub(X-Hub-Signature-256) · Stripe(Stripe-Signature) · Slack(X-Slack-Signature) 모두 HMAC-SHA256 채택. 라이브러리와 레퍼런스 풍부.

VASP 미지원 시 대응 전략

AUTHENTICATION METHOD CHANGES — ONLY ONE PLACE IN CODE

Phase별 VASP 인증 방식

- 월렛원 (Phase 1) — 계약 협의 필요 (HMAC or 자체 토큰)
- Circle (Phase 2) — Circle 자체 서명 방식
- 구형 VASP — IP 화이트리스트만 지원
- 고보안 구간 — mTLS (클라이언트 인증서)

HMAC 미지원 시 대체 전략

- IP 화이트리스트 + TLS 조합
- VASP 자체 토큰 방식 적용
- mTLS — 클라이언트 인증서 상호 검증
- Bearer Token (API Key) 방식

어떤 방식을 써도 코드 변경 범위는 동일하다

```
// 인증 방식 교체 시 변경 위치: _verifySignature() 내부 한 곳
private _verifySignature(rawBody: Buffer, sig: string): boolean {
  // HMAC → mTLS → VASP 자체 토큰 ... 어떤 방식이든 여기만 바꾼다
  const expected = crypto.createHmac('sha256', this.config.secret).update(rawBody).digest('hex');
  ...
}
// WebhookServer · RedisStreamPublisher · ConsumerGroupWorker → 한 줄도 수정 없음
```

M8에서 확인 예정 — EIP-712 SafeTx (비대칭 서명)를 다룰 때 이 구조의 차이가 명확하게 드러난다.

금융 시스템 추가 고려사항

BEYOND BASIC HMAC — WHY GENERAL WEBHOOK VALIDATION IS NOT ENOUGH

① Secret 로테이션 정책

- 유출 의심 시 즉시 교체 가능해야 함
- 교체 과도기(rollover): 현재 키 + 이전 키 동시 검증 → 무중단 교체
- 교보 프로젝트: 90일 주기 자동 교체
- AWS Secrets Manager 또는 Vault 사용

② Timestamp 검증

- Replay Attack 방어 목적
- 요청 timestamp ± 5 분 이내만 허용
- 이 코드에는 없음
- → S9 IdempotencyGuard가 requestId로 보완

③ TLS 전제 — HMAC은 TLS(HTTPS) 위에서만 의미 있음

HTTP에서는 서명이 맞아도 네트워크 도청으로 replay 가능 · 교보 프로젝트: 모든 Webhook 통신 TLS 1.3 이상 필수

Secret이 유출되면? 공격자가 임의 payload에 올바른 서명 생성 가능 → 검증 무의미. 코드 하드코딩 절대 금지 · `process.env.WEBHOOK_SECRET` 또는 Vault 사용.

HMAC 원리 — 송수신 대칭 구조

HASH-BASED MESSAGE AUTHENTICATION CODE

발신자 (교보 앱 서버)

```
secret = "kyobo-webhook-secret-2024"
payload = '{eventType:"ACTIVITY_ACHIEVED",...}'

sig = HMAC-SHA256(secret, payload)
→ "a3f4b2c1d8e9f0..." (64자 hex)
```

HTTP 요청:

X-Kyobo-Signature: a3f4b2c1d8e9f0...

Body: {eventType:"ACTIVITY_ACHIEVED",...}



수신자 (WebhookServer)

```
rawBody = 수신한 Body bytes (Buffer 그대로)
expected = HMAC-SHA256(secret, rawBody)
← 발신자와 동일한 계산

timingSafeEqual(received, expected)
```

일치 → 발신자 인증 + 무결성 ✓

불일치 → 위조된 요청 ✗

송신과 수신 — 거울 대칭

SENDER AND RECEIVER ARE MIRROR IMAGES

	송신 (외부 클라이언트)	수신 (이 코드)
입력	payload 객체	rawBody Buffer
직렬화	JSON.stringify(obj)	(이미 직렬화된 상태)
바이트화	Buffer.from(json)	(이미 Buffer)
HMAC 알고리즘	SHA-256	SHA-256
Secret	'kyobo-webhook-secret-2024'	this.config.secret
출력	hex 문자열 → X-Kyobo-Signature 헤더	hex 문자열 → 비교

같은 secret + 같은 알고리즘 + 같은 입력
→ 같은 출력 → 정당한 요청 ✓

secret이 다르거나 body가 바뀌면
→ 결과 완전히 달라짐 → 위조 ✗

Timing Attack — 왜 === 로 비교하면 안 되는가

RESPONSE TIME LEAKS SIGNATURE INFORMATION — ns LEVEL DIFFERENCE

❌ 일반 비교의 문제

```
// 내부: 첫 번째 다른 바이트에서 즉시 false  
received === expected
```

공격자 전략:

"0000...0000" → 0.1ms (첫 바이트 틀림)

"a000...0000" → 0.2ms (두 번째에서 틀림)

"a300...0000" → 0.3ms (세 번째에서 틀림)

...

→ 시간 패턴으로 서명을 한 자리씩 추측

✅ timingSafeEqual

```
crypto.timingSafeEqual(sigBuf, expBuf)
```

- 항상 모든 바이트를 끝까지 비교
- 첫 바이트 다른 마지막이 다른
항상 동일한 시간 소요
- 응답 시간으로 서명 추측 불가

// 현실: 네트워크 지연이 ns 차이를 가리지만

// 보안 표준은 항상 `timingSafeEqual` 권장

주의: `timingSafeEqual`은 두 Buffer 길이가 다르면 `TypeError: Input buffers must have the same byte length throw` — 반드시 길이 사전 검사 필요

TS 문법 — 이 코드에서 새로 등장한 것

TYPESCRIPT SYNTAX HIGHLIGHTS IN `_verifySignature`

```
private _verifySignature(...): boolean
```

- `private + _ prefix` → 내부 전용 메서드 (외부 호출 불가)
- 반환 타입 : `boolean` → `true/false`만 리턴
- `Buffer` → Nodejs 내장 타입, 바이트 배열

```
private _verifySignature(  
  requestBody: Buffer, signature: string  
): boolean { ... }
```

```
Buffer.from(str, 'hex')
```

- 두 번째 인자 = 인코딩 지정
- `'hex'` → 두 글자씩 1바이트로 해석
- `'a3f2'` (4글자) → `[0xa3, 0xf2]` (2바이트)
- `.length`는 바이트 수 (글자 수 아님)
- SHA-256 = 항상 32바이트 = hex 64글자

```
// 64글자 hex → 32바이트 Buffer  
Buffer.from(signature, 'hex')  
Buffer.from(expected, 'hex')
```

다른 인코딩: `'utf8'` · `'base64'` · `'binary'` — `hex`는 "바이트를 16진수 문자열로 표현"

두 가지 핵심 설계 포인트

rawBody BUFFER · LENGTH CHECK BEFORE timingSafeEqual

포인트 ①: rawBody Buffer 그대로 계산

```
// ❌ 재직렬화 함정
const body = JSON.parse(rawBody.toString());
const re = JSON.stringify(body);
.update(re) // 키 순서·공백 변경 가능 → 불일치

// 예: 발신자 '{"a":1,"b":2}'
// JS 재직렬화 → 같을 수도, Python과 다를 수도
// 한 바이트만 달라도 HMAC이 완전히 달라짐

// ✅ 원본 Buffer 그대로
.update(rawBody) // 발신자와 동일한 바이트
```

포인트 ②: 길이 사전 검사

```
// ❌ 길이 검사 없이 바로
crypto.timingSafeEqual(sigBuf, expBuf);
// → TypeError 가능 (길이 다를 때 throw)

// ✅ 사전 차단
if (sigBuf.length !== expBuf.length)
  return false;
crypto.timingSafeEqual(sigBuf, expBuf);

// SHA-256 = 항상 32바이트 = 공개 정보
// 길이 비교 자체는 timing safe 불필요
```

핵심: 발신자가 서명할 때 쓴 바이트 그대로 수신 측이 재계산해야 일치. JSON 재직렬화는 그 바이트를 바꿀 수 있다.

_verifySignature 라인별 흐름

STEP-BY-STEP CODE WALKTHROUGH — 5 LINES, 5 DECISIONS

단계	코드	의미 · 설계 이유
①	<code>if (!signature) return false</code>	헤더 자체 없으면 즉시 거부. 빈 문자열·undefined·null 모두 falsy로 처리
②	<code>createHmac('sha256', secret).update(rawBody).digest('hex')</code>	서버 측에서 동일한 계산 수행. rawBody Buffer 그대로 — JSON 재직렬화 함정 회피
③	<code>Buffer.from(signature, 'hex')</code> <code>Buffer.from(expected, 'hex')</code>	<code>timingSafeEqual</code> 은 Buffer만 받음 → hex 문자열을 바이트로 변환. 64글자 → 32바이트
④	<code>if (sigBuf.length !== expBuf.length) return false</code>	<code>timingSafeEqual</code> 은 길이 다르면 throw. SHA-256 길이는 공개 정보이므로 일반 비교로 충분
⑤	<code>return crypto.timingSafeEqual(sigBuf, expBuf)</code>	모든 바이트를 끝까지 비교 → Timing Attack 방지. 결과는 boolean

각 단계가 막는 공격: ① 헤더 누락 · ② JSON 재직렬화 불일치 · ④ throw + prefix-match · ⑤ Timing Attack

_verifySignature() 완성본

COMPLETE IMPLEMENTATION — ANNOTATED

```
private _verifySignature(rawBody: Buffer, signature: string): boolean {
  // ㉠ 서명 헤더 없음 → 즉시 false
  if (!signature) return false;

  // ㉡ rawBody Buffer로 HMAC-SHA256 계산 (JSON 재직렬화 함정 회피)
  const expected = crypto
    .createHmac('sha256', this.config.secret)
    .update(rawBody)           // ← string이 아닌 Buffer 그대로
    .digest('hex');           // ← hex string 반환

  // ㉢ hex → Buffer 변환 (byte-level 비교를 위해)
  const sigBuf = Buffer.from(signature, 'hex'); // 64글자 → 32바이트
  const expBuf = Buffer.from(expected, 'hex');

  // ㉣ 길이 불일치 → timingSafeEqual이 throw하므로 사전 차단
  if (sigBuf.length !== expBuf.length) return false;

  // ㉤ 항상 동일 시간에 비교 — Timing Attack 방지
  return crypto.timingSafeEqual(sigBuf, expBuf);
}
```

WebhookServer 전체 구조

`_verifySignature` IS THE GATE INSIDE `_handleRequest`

```
export class WebhookServer {
  constructor(private readonly config: {
    port: number; secret: string; maxBodyKb: number; // ← secret은 환경변수로 주입
  }) { ... }

  on(eventType: string, handler: WebhookHandler): this { ... } // 핸들러 등록

  private async _handleRequest(req, res): Promise<void> {
    const rawBody = await this._readBody(req);
    const signature = req.headers['x-kyobo-signature'] as string ?? '';

    if (!this._verifySignature(rawBody, signature)) {
      res.writeHead(401).end('invalid signature');
      return;
    }

    const payload = JSON.parse(rawBody.toString('utf8'));
    res.writeHead(202).end(); // ← 즉시 응답

    const handlers = this.handlers.get(payload.eventType) ?? [];
    await Promise.allSettled(handlers.map(h => h(payload))); // ← 비동기
  }

  private _verifySignature(rawBody: Buffer, signature: string): boolean { ... }
}
```

보안 체크리스트 & 강의 강조 포인트

WHAT THIS CODE COVERS — AND WHAT IT DOESN'T

항목	처리	방식
헤더 누락	✓	즉시 false
JSON 재직렬화 함정	✓	rawBody Buffer 사용
길이 불일치 throw	✓	사전 길이 검사
Timing Attack	✓	timingSafeEqual
Replay Attack	✗	S9 IdempotencyGuard
Secret 유출 방지	✗	환경변수/Vault 분리

꼭 기억할 포인트

- rawBody는 절대 JSON.parse 후 stringify 금지
- secret은 코드 하드코딩 금지 → env/Vault
- timingSafeEqual은 길이 다르면 throw → 사전 검사
- Buffer.from(hex, 'hex') → 바이트 변환 표준
- 송수신 거울 대칭 — 같은 알고리즘 + secret
- 금융권: 90일 주기 Secret 로테이션
- HMAC만으론 부족 → Replay Attack 별도 방어

완성된 파이프라인 — main.ts 초기화

S5 + S6 COMBINED — WebhookPublishHandler WIRES EVERYTHING

```
import { WebhookServer }      from './webhook/WebhookServer';
import { WebhookPublishHandler } from './webhook/WebhookPublishHandler';
import { IdempotencyGuard, InMemoryIdempotencyStore } from './webhook/IdempotencyGuard';
import { RedisStreamPublisher } from './dmz/RedisStreamPublisher';

const publisher = new RedisStreamPublisher(redis);
await publisher.initialize();      // XGROUP CREATE (BUSYGROUP 무시)

const idempotency = new IdempotencyGuard(new InMemoryIdempotencyStore());
// 운영: new RedisIdempotencyStore(redis)

const server = new WebhookServer({
  port: 3000, secret: process.env.WEBHOOK_SECRET!, maxBodyKb: 64,
});

const handler = new WebhookPublishHandler(publisher, idempotency);
server.on('NFT_ISSUED', handler.createHandler());
// createHandler()가 반환한 함수 → idempotency.run() → publisher.publish()

await server.listen();
console.log('[app] 이벤트 수신 파이프라인 ready');
```

WebhookPublishHandler 역할: IdempotencyGuard.run(requestId) → 중복 차단 · RedisStreamPublisher.publish() → XADD · 인라인 람다 대신 클래스로 분리 → 단위 테스트 가능

실습 Step 1 — _verifySignature 직접 구현

TODO → ANSWER · 15MIN · [exercises/S06_hmac_webhook.ts](#)

```
private _verifySignature(rawBody: Buffer, signature: string): boolean {
  // TODO 1: signature가 없으면 즉시 false 반환

  // TODO 2: crypto.createHmac으로 expected 계산
  //   - algorithm: 'sha256'
  //   - key: this.config.secret
  //   - data: rawBody (Buffer 그대로, string 변환 금지)
  //   - 결과: hex string (.digest('hex'))

  // TODO 3: signature, expected를 각각 Buffer.from(?, 'hex') 로 변환

  // TODO 4: 길이가 다르면 false 반환
  //   (timingSafeEqual이 길이 다르면 throw하기 때문)

  // TODO 5: crypto.timingSafeEqual로 비교 결과 반환

  return false; // placeholder - 구현 후 제거
}
```

실행: `npx ts-node src/exercises/S06_hmac_webhook.ts` (event-engine 패키지 루트에서)

실습 Step 2 — 서버 기동 & 서명 생성

TEST SERVER ON :3002 · generate-sig.js

테스트 서버 기동

```
const SECRET = 'kyobo-test-secret-2024';
const server = new WebhookServer({
  port: 3002, secret: SECRET, maxBodyKb: 64
});
server.on('ACTIVITY_ACHIEVED', async (p) => {
  console.log('[received]', p.requestId);
});
await server.listen();
```

서명 생성 (generate-sig.js)

```
const crypto = require('crypto');
const payload = JSON.stringify({
  eventType: 'ACTIVITY_ACHIEVED',
  data: { userId: 'u-001' },
  timestamp: 1714000000,
  requestId: 'test-correct-sig',
});
const sig = crypto
  .createHmac('sha256', 'kyobo-test-secret-2024')
  .update(Buffer.from(payload)) // ← Buffer.from() 필수
  .digest('hex');
console.log('SIG:', sig);
```

핵심: 서명 생성 시에도 `Buffer.from(payload)` — JSON 문자열을 Buffer로 변환해야 `rawBody`와 동일한 바이트로 계산됨

실습 테스트 시나리오 1 ~ 3

curl TEST · EXPECTED: 401 / 401 / 202

시나리오 1: 서명 없음 → 401

```
curl -s -o /dev/null \  
-w "%{http_code}\n" \  
-X POST http://localhost:3002 \  
-H "Content-Type: application/json" \  
-d '{"eventType":  
  "ACTIVITY_ACHIEVED",  
  "requestId":"no-sig"}'
```

예상: 401

시나리오 2: 잘못된 서명 → 401

```
curl -s -o /dev/null \  
-w "%{http_code}\n" \  
-X POST http://localhost:3002 \  
-H "Content-Type: application/json" \  
-H "X-Kyobo-Signature: \  
  0000...0000" \  
-d '{"eventType":  
  "ACTIVITY_ACHIEVED",  
  "requestId":"bad-sig"}'
```

예상: 401

시나리오 3: 올바른 서명 → 202

```
SIG=$(node generate-sig.js)  
  
curl -s -o /dev/null \  
-w "%{http_code}\n" \  
-X POST http://localhost:3002 \  
-H "Content-Type: application/json" \  
-H "X-Kyobo-Signature: $SIG" \  
-d "$PAYLOAD"
```

예상: 202

세 가지 결과 401 / 401 / 202를 모두 확인해야 _verifySignature 구현이 올바른 것

실습 시나리오 4 — JSON 재직렬화 함정 (심화)

WHY `rawBody` MATTERS — CROSS-LANGUAGE SERIALIZATION MISMATCH

```
// JSON 재직렬화 함정 시뮬레이션
const original = '{"b":2,"a":1}'; // 발신자 원본 (키 순서: b, a)

// 발신자 서명 (원본 바이트 그대로)
const originalSig = crypto.createHmac('sha256', SECRET)
    .update(Buffer.from(original)).digest('hex');

// 수신자가 재직렬화해서 검증하면?
const parsed = JSON.parse(original); // { b: 2, a: 1 }
const reserialized = JSON.stringify(parsed); // '{"b":2,"a":1}' ← V8은 순서 보존하지만 보장 없음

// 실제 문제가 되는 케이스:
// 발신자: Python dict → '{"a":1,"b":2}' (알파벳 순 정렬)
// 수신자: JS JSON.stringify → '{"b":2,"a":1}' (삽입 순서)
// → 서명 완전 불일치 ❌

// 결론: rawBody Buffer 그대로 서명 계산만이 100% 안전
```

실무 주의: 발신자가 Python · Go · Java인 경우 JSON 직렬화 순서가 JS와 다를 수 있음. `rawBody` Buffer만이 유일하게 안전한 방법.

S5 ~ S6 완성 파이프라인

FULL EVENT PIPELINE — WEBHOOK → VERIFY → 202 → QUEUE → WORKER

외부 (교보 앱 / VASP)

내부망 — 전체 파이프라인

내부망 — Webhook 수신

S5 · S6

① `_readBody()`

rawBody Buffer 수집

② `_verifySignature()` ★

HMAC-SHA256 · timingSafeEqual

false → 401 true → 계속

③ `res.writeHead(202).end()`

즉시 응답 — 발신자 timeout 방지

비동기
▶

큐 적재 — 핸들러

S7 · S9

WebhookPublishHandler

handler(payload) 비동기 실행

IdempotencyGuard.run()

중복 requestId → 스킵 (S9)

RedisStreamPublisher.publish()

XADD kyobo:events * (S7)

Redis Streams: kyobo:events

append-only 로그

XREADGRO
▶

컨슈머 처리

S10 · S12

ConsumerGroupWorker

XREADGROUP + XACK (S10)

NFTIssuedProcessor

먹등성 재확인 → 원장 업데이트 (S12)

LedgerService.creditNFT()

DB 트랜잭션 → XACK

읽는 법 — 외부 요청이 내부망에 도달 → `_readBody` · `_verifySignature(HMAC)` → 202 즉시 응답 → 비동기 큐 적재 → Consumer Worker가 먹등성 처리 후 원장 반영.

핵심 정리 & 완료 기준 체크리스트

KEY CONCEPTS · COMPLETION CRITERIA

개념	구현	이유
HMAC-SHA256	<code>createHmac('sha256', secret).update(rawBody).digest('hex')</code>	위변조 거부
rawBody Buffer	<code>.update(rawBody)</code> — <code>JSON.stringify</code> 금지	직렬화 불일치 방지
timingSafeEqual	<code>crypto.timingSafeEqual(sigBuf, expBuf)</code>	Timing Attack 방지
길이 사전 체크	<code>if (sigBuf.length !== expBuf.length) return false</code>	throw 방지

완료 기준

- `_verifySignature` TODO → 직접 구현 완료
- curl 3시나리오: 401 / 401 / 202 모두 확인
- rawBody Buffer 이유 설명 가능
- timingSafeEqual throw 조건 설명 가능
- S5~S6 전체 파이프라인 코드 레벨 설명 가능
- WebhookPublishHandler 클래스 분리 이유 설명

Block A 세션 흐름

- **S5:** 202 패턴 · Webhook 즉시 응답
- **S6(S8):** `_verifySignature` · HMAC
- **S7:** Redis Streams 이론 · Consumer Group
- **S8:** Redis CLI 실습 · XADD/XREADGROUP
- **S9:** At-least-once · IdempotencyGuard
- **S10:** ConsumerGroupWorker 코드 상세

HMAC은 VASP 전용인가? — 온체인 이벤트와의 신뢰 모델 비교

TRUST MODEL · HMAC vs ON-CHAIN VERIFICATION

질문: HMAC은 웹훅 발신자가 VASP일 때만 필요한가? 나중에 온체인에서 직접 이벤트를 받으면 어떻게 신뢰를 보장하나?

신뢰 문제	HTTP 웹훅 (VASP → Issuer)	온체인 이벤트 (체인 → EventEngine)
발신자 인증	HMAC 서명 검증	트랜잭션 서명 (ECDSA) — 합의로 검증됨
데이터 무결성	HMAC-SHA256	Merkle Proof + 블록 합의
재전송 공격 방지	timestamp + nonce	블록 번호 · 트랜잭션 순서 보장
신뢰 기반	공유 시크릿 (사전 교환)	블록체인 프로토콜 자체

온체인에서 대신 검증하는 것들

- 컨트랙트 주소 확인 — 알려진 주소의 이벤트만 처리
- Finalized 블록 대기 — reorg 방지 (S12)
- 노드 신뢰 — 자체 노드 또는 인증된 RPC (Alchemy/Infura API Key + TLS)

핵심 원칙

HMAC은 "HTTP 발신자를 못 믿는 문제"를 푸는 도구.
온체인 이벤트는 블록체인 합의 알고리즘이 그 역할을 대체한다.

아키텍처가 바뀌면 신뢰 보장 방식도 바뀐다.

예상 Q&A

ANTICIPATED QUESTIONS — THREE COMMON ONES

Q1. 서명 시크릿이 환경변수로 노출되면?

공격자가 임의 payload에 올바른 서명 생성 가능 → 검증 무의미. 코드 하드코딩 절대 금지 · 30~90일 로테이션 · 노출 의심 시 즉시 교체 + 로그 분석. 교보: AWS Secrets Manager / Vault.

Q2. publish()가 throw하면 이벤트가 유실되는가?

202는 이미 전송됐으므로 발신자는 "성공" 인식 → 유실 가능. 방어: ① Redis HA 구성 ② publish 실패 시 인메모리 재시도 큐 ③ ChainEventListener._recoverMissedEvents로 체인 재스캔 보완.

Q3. Content-Type이 application/json이 아닌 요청이 오면?

현재 WebhookServer는 Content-Type 미검증. 서명 통과 후 JSON.parse에서 400 반환. 명확한 처리 추가:

```
if (req.headers['content-type'] !== 'application/json') { res.writeHead(415).end(); return; }
```